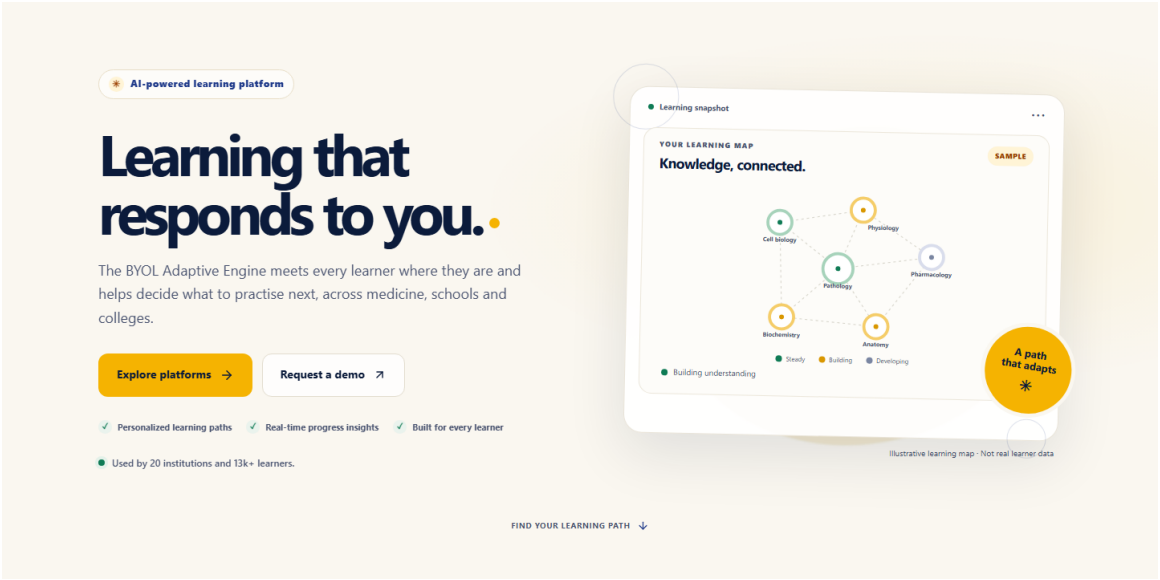
BYOL ACADEMY

Learn web development through this website

A detailed beginner's guide to how a website works, how this project is organized, what every project file does, and how to safely make your first changes.



The finished homepage: we will trace how its text, components, styles and route combine.

> A learning companion, not a programming exam
> You do not need to memorize the code. Read the diagrams, follow one file at a time in VS Code, and use the exercises near the end to learn by changing small things.

This guide is based on the BYOL Academy website source in E:\WORK\Byol Academy. It describes the files that exist in this project; it is not a general Next.js manual.

How to use this book

Part	Goal
1-3. Foundations	Understand what happens between typing a website address and seeing a page.
4-6. Project map	Learn the folders, routes, content and reusable components.
7-9. Real features	Follow the interactive learning demo and the contact form from screen to server.
10-12. Quality and delivery	Understand CSS/accessibility, tests, CI and settings.
13-15. File catalog and practice	Know what each source/configuration file is for and practice changing the site.

First things to install

A code editor (VS Code), Node.js 22 LTS and npm are used here. The project files are already present in the workspace. In PowerShell, run commands from the project folder so npm can find package.json.

```
cd "E:\WORK\Byol Academy"
npm install
npm run dev
```

Then open http://localhost:3000 in a browser. Keep the PowerShell window open while developing. Press Ctrl+C in that terminal to stop the local server.

> What does “repository” mean?
> People often call the project's main folder a repository. This workspace contains the project files, but Git repository tracking is not configured in the current folder. It can still run locally. Git is an optional version-history tool, not the website itself.

1. How a website works - from address to screen

A web page is not one file. Several pieces cooperate: your browser asks for a URL, a web server chooses a route, the application creates a response, and the browser draws it.

```
You type https://byolacademy.com/about
        |
        v
Browser asks the website server for /about
        |
        v
Next.js selects src/app/about/page.tsx
        |
        +--> React creates the page structure
        +--> content files provide text and product data
        +--> CSS rules provide colors, spacing and layout
        |
        v
Server sends page output to the browser
        |
        v
Browser displays it; interactive pieces respond to clicks
```

The basic browser languages

Technology	Job	A familiar comparison
HTML	Names and organizes page elements: headings, links, buttons, forms.	The structure and labels of a room.
CSS	Controls colors, size, spacing, layout, and responsive behavior.	Paint, furniture, lighting, arrangement.
JavaScript	Adds behavior: clicking, opening a menu, submitting a form.	The switches and moving parts.
TypeScript	JavaScript plus rules describing what values are allowed.	A checklist that catches some mistakes before opening.
React	Lets us build interfaces from reusable components and state.	Reusable building blocks.
Next.js	Framework that connects React pages, URLs, server code, SEO and builds.	The site construction system.

What is a server request?

HTTP is the request-and-response conversation between a browser and a server. A GET request usually asks to view a page. A POST request usually sends data, such as an enquiry. The project has ordinary page routes and one server route, /api/lead, for possible enquiries.

What does it mean to render?

Rendering means turning a component description into page output. Next.js can render much of this site's content on the server, then send the result to the browser. Interactive components also run JavaScript in the browser. This combination helps pages show useful content before a visitor interacts.

> A URL has parts
> In /contact?intent=institution, /contact selects the page and ?intent=institution carries optional context. The contact page reads that query value and can prepare the form for an institution enquiry.

2. React components, TSX, props and state

Many project files end in .tsx: they contain TypeScript plus JSX, a syntax that lets code describe HTML-like page structure.

A component is a reusable function

```
function Welcome() {
  return <h1>Welcome to BYOL Academy</h1>;
}
```

The function returns a description of what should appear. In React, component names start with a capital letter. The app can place it as <Welcome />.

JSX puts values into page structure

```
const learner = "Asha";
return <p>Hello, {learner}!</p>;
```

Curly braces mean “evaluate this JavaScript expression here.” JSX resembles HTML, but it is written inside JavaScript/TypeScript. For example, CSS classes use className rather than HTML's class attribute.

Props are inputs; state is remembered interaction

```
type GreetingProps = { name: string };

function Greeting({ name }: GreetingProps) {
  return <p>Hello, {name}!</p>;
}

<Greeting name="Asha" />
```

The Greeting component receives a prop named name. Its type says the value must be text. State differs: React remembers state inside an interactive component and redraws it when it changes. The mobile menu's open/closed value and the demo's current question are state.

Lists and stable keys

```
platforms.map((platform) => (
  <article key={platform.id}>
    <h3>{platform.name}</h3>
  </article>
))
```

map() loops through a list and returns one card for each item. A key uniquely identifies each card to React. This is why product copy can live in data while the component renders all cards consistently.

Imports connect files

```
import { AdaptiveDemo } from "@/components/AdaptiveDemo";
import { platforms } from "@/content/site";
```

import makes an exported value available in this file. The @/ prefix is a shortcut configured in tsconfig.json for the src folder. Without it, the path might need several ../ segments.

> TypeScript errors are useful messages
> When a function expects a string but receives a number, TypeScript can warn before the browser runs the code. `strict: true` in tsconfig.json asks for thorough checks.

3. Project map: where things live

The project is organized by responsibility. Find the route, reusable UI, content, helper logic, styles, tests, or root configuration depending on what you need to change.

```
Project folder
|
+-- src/
|   +-- app/          URL pages and server route
|   +-- components/   reusable user-interface parts
|   +-- content/      wording, products and demo questions
|   +-- lib/          calculations and validation helpers
|   +-- styles/       global design rules
|
+-- tests/
|   +-- unit/         small logic checks
|   +-- e2e/          real-browser checks
|
+-- .github/workflows/ automated CI checks
+-- package.json       project commands and libraries
+-- configuration files, documentation and supplied briefs
```

Folder	What belongs there	Common reason to open it
src/app	Each page route and server endpoints.	Add or change a website page.
src/components	Reusable navigation, form, demo and map UI.	Change interaction or shared visual piece.
src/content	Text and structured lists.	Edit wording or sample data.
src/lib	Pure logic and server helpers.	Change calculations or validation.
src/styles	Global CSS/design system.	Adjust colors, spacing or responsive layout.
tests	Automated expected behavior.	Check a code change stays correct.
.github/workflows	Automation recipe for GitHub Actions.	Understand remote code checks.

Generated folders are different

Folder/file	Meaning	Should you edit it?
node_modules	Downloaded third-party libraries.	No. npm manages it.
.next	Next.js generated build/development output.	No. It can be regenerated.
tsconfig.tsbuildinfo	TypeScript incremental-check cache.	No.
playwright-report / test-results	Generated browser-test output when tests run.	Usually no; inspect results, do not hand-edit.

Don't start by opening node_modules
The project code you maintain is mainly in src/, tests/ and the root configuration/docs. Third-party package internals are not the place to make your feature changes.

4. How folders become website addresses

Next.js App Router uses folders and special filenames. A page.tsx file creates the user-facing route at its folder path.

Website address	Source file	What the visitor sees
/	src/app/page.tsx	Homepage with products, audiences and interactive demo.
/about	src/app/about/page.tsx	BYOL approach, research context and principles.
/blog	src/app/blog/page.tsx	Blog empty state until articles are approved.
/contact	src/app/contact/page.tsx	Contact information and gated enquiry form.
/privacy	src/app/privacy/page.tsx	Privacy-policy placeholder, not a legal policy.
/terms	src/app/terms/page.tsx	Terms placeholder, not legal terms.
/styleguide	src/app/styleguide/page.tsx	Internal visual sample; excluded from indexing.
Unknown URL	src/app/not-found.tsx	Friendly branded 404 page.
Rendering error	src/app/error.tsx	Retry and return-home screen.
POST /api/lead	src/app/api/lead/route.ts	Server checks and possible enquiry delivery.

Example: the About page

```
src/app/about/page.tsx
  export const metadata = { title: "About BYOL Academy", ... };
  export default function AboutPage() {
    return (
      <>
        <section>Page introduction</section>
        <section>Conceptual learning loop</section>
        <section id="research">Research</section>
        <section id="responsible-ai">Principles</section>
      </>
    );
  }
```

The metadata sets the page title and description. The default exported function returns the page. section elements group related content. The #research URL fragment scrolls to the section whose id is research.

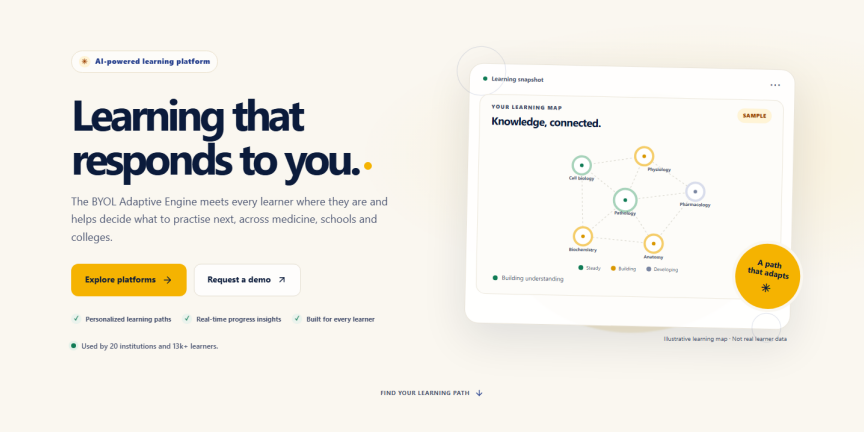
Route special files

Filename	Role
page.tsx	Renders a route's page.
layout.tsx	Wraps routes in shared UI.
error.tsx	Handles an error within a route segment; offers retry.
not-found.tsx	Renders for missing routes/content.
route.ts	Handles HTTP methods such as POST; returns an HTTP response.
robots.ts / sitemap.ts	Generate standard search crawler files.

Why legal pages are placeholders
The business must supply approved legal wording. The site says this honestly and keeps enquiry collection disabled rather than inventing policies.

5. How the homepage and content data connect

The homepage is assembled in page.tsx, but many of its words and lists are imported from site.ts. This separation makes wording easier to locate than burying it in layout markup.

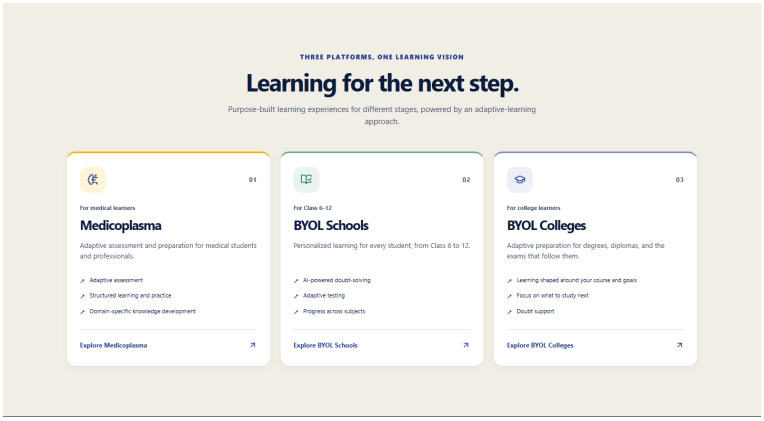


The rendered hero section corresponds to the opening markup in src/app/page.tsx.

```
import { platforms, copy, shippable } from "@/content/site";

export default function HomePage() {
  const visiblePlatforms = shippable(platforms);
  return (
    <section id="platforms">
      {visiblePlatforms.map((platform) => (
        <article key={platform.id}>
          <h3>{platform.name}</h3>
          <p>{platform.summary}</p>
          <a href={platform.href}>{platform.ctaLabel}</a>
        </article>
      ))}
    </section>
  );
}
```

The imports bring in site copy and platform data. `shippable()` filters out unapproved entries in production. The loop builds a card from each approved record. The same platform list is reused in navigation/footer/contact so links remain consistent.



Three platform records use the same card layout with different content.

How the content type helps

Platform is a shape contract in site.ts. Each record needs an ID, product name, audience, summary, feature list, URL, button label and approval status. If a developer forgets a required field, TypeScript reports it.

Don't invent business claims

The content source contains approved items and explicit responsible-AI boundaries. The site withholds customer logos/testimonials when permission is missing, and does not describe the sample demo as BYOL's deployed engine.

6. Reusable components: interface building blocks

A component keeps a piece of UI in one place so it can be reused, tested and understood independently.

Component	What it does	Beginner concept
Header.tsx	Navigation, mobile menu, product dropdown, request-demo button.	State + click/keyboard events.
AnnouncementBar.tsx	Announcement visitors can dismiss for the browser tab session.	Browser storage + external-store subscription.
Footer.tsx	Reusable links at the bottom of each route.	Data-driven repeated links.
AdaptiveDemo.tsx	Question flow and sample results.	State machine + event handlers.
MasteryMap.tsx	SVG concept visualization and equivalent data table.	Props + SVG + accessibility.
LeadForm.tsx	Form controls, validation feedback, request to API.	Controlled fields + async request.
AnalyticsEvents.tsx	Optional click/scroll tracking.	Browser event listeners.
ApprovalNotice.tsx	Development marker for unapproved content.	Environment-specific rendering.
JsonLd.tsx	Structured descriptions for search engines.	HTML metadata.

Client components and the “use client” line

Header, AnnouncementBar, AdaptiveDemo, LeadForm and AnalyticsEvents use browser interactions. Their first line, "use client", marks them as client components. A static page such as About can remain server-rendered because it does not need its own click state.

A menu's open/closed value

```
const [menuOpen, setMenuOpen] = useState(false);

<button
  aria-expanded={menuOpen}
  onClick={() => setMenuOpen((open) => !open)}
>
  Open menu
</button>
```

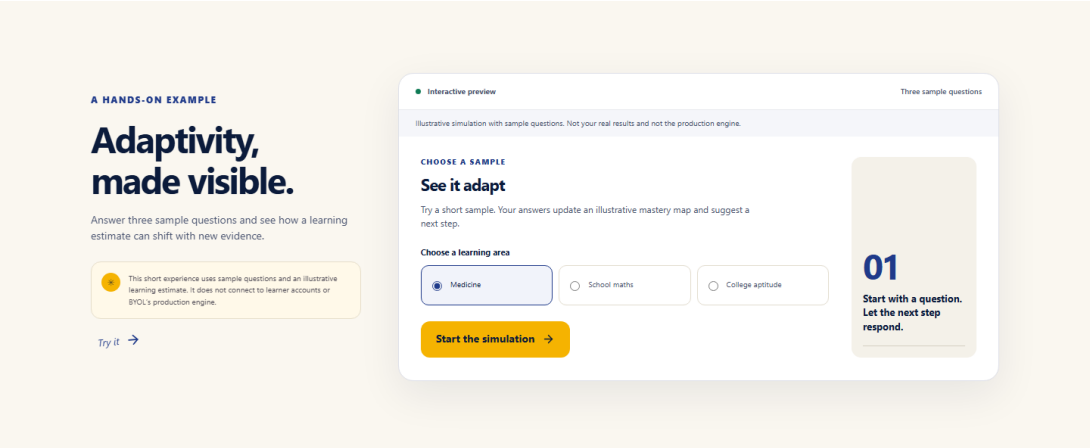
useState(false) starts the menu closed. The setter receives the previous value and flips it. React redraws the component with the updated value. aria-expanded communicates that state to assistive technology. The real Header also closes with Escape and returns keyboard focus.

The announcement uses sessionStorage

When dismissed, the announcement saves a small true/false value in sessionStorage. That remembers the choice for this tab session; it is not a user account and contains no personal data. The component uses React's external-store hook to stay in sync with the saved state.

7. Follow the learning demo from click to result

This is the best example of interactive front-end code in the project. The questions, display state, estimate calculation and visual/text results are kept in separate files.



The demo begins in idle state, waiting for a subject and a start action.

The four-state flow

State	Screen	Action that changes state
idle	Choose Medicine, School maths or College aptitude.	Start -> asking
asking	Show prompt, answer choices and progress.	Choose/check -> feedback
feedback	Show whether the answer was right and explain why.	Next -> asking or results
results	Show concept estimates, map, table and next-step text.	Restart -> idle

```
type DemoState = "idle" | "asking" | "feedback" | "results";
const [state, setState] = useState<DemoState>("idle");
const [questionIndex, setQuestionIndex] = useState(0);
const [selected, setSelected] = useState<number | null>(null);
const [estimates, setEstimates] = useState<number[]>([]);
```

The allowed state strings are written as a TypeScript union. `questionIndex` starts at zero because arrays count from zero. `selected` is null until the learner chooses an answer. `estimates` starts empty and grows as answers are checked.

Question data is separate from the component

src/content/demo-data.ts contains three subjects, each with three question objects. Each object has a concept, prompt, option list, correct answer index and explanation. This separation means content editors can update sample wording without rewriting the flow.

Submit: compare, calculate and show feedback

```
function submitAnswer() {
  if (selected === null || !question) return;
  const correct = selected === question.answer;
  const estimate = updateMastery(0.5, correct);
  setEstimates((previous) => [...previous, estimate]);
  setCorrectCount((previous) => previous + Number(correct));
  setState("feedback");
}
```

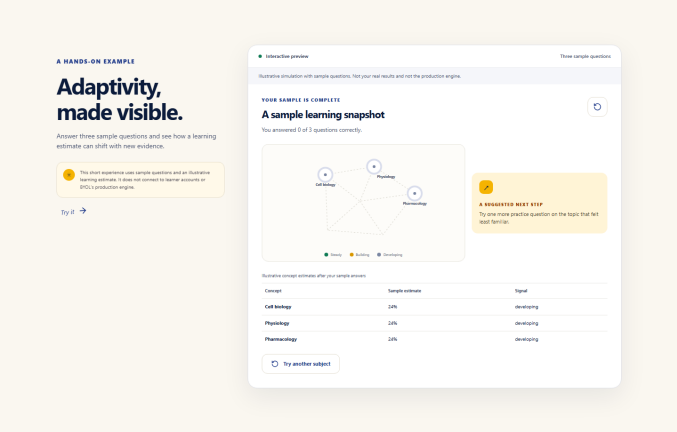
The first line prevents processing without a choice/question. `correct` is a true/false comparison. The mastery helper computes a sample probability. The `...previous` syntax copies old estimates into a new list and appends one. Number(true) is 1 and Number(false) is 0, so the count only increases for a correct answer.

What is the probability calculation?

The helper in src/lib/adaptive/mastery.ts is an illustrative Bayesian Knowledge Tracing-style calculation. It begins with a prior estimate, allows that a learner may guess or slip, uses the answer as evidence to calculate a posterior, then adds a small learning probability. The sample values are guess 0.20, slip 0.10 and learn 0.15.

```
// If prior=0.50 and the answer is correct:
likelihood = 0.50 * (1 - 0.10) = 0.45
guess evidence = (1 - 0.50) * 0.20 = 0.10
evidence total = 0.45 + 0.10 = 0.55
posterior = 0.45 / 0.55 = about 0.818
add learning chance -> about 0.845
```

The helper rejects values outside valid probability bounds with RangeError. masterySignal() converts a numeric estimate into the readable label developing, building or steady. The demo starts each question from the same 0.5 prior; it is not a production adaptive sequencing system.

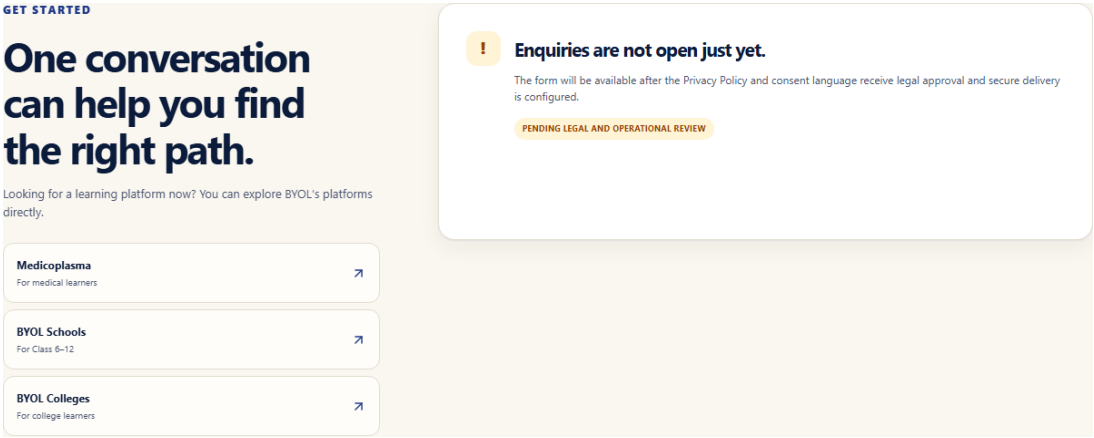


After all sample questions, the demo shows a visual map and an equivalent readable table.

> The key honesty rule
> This is local demonstration math using sample questions. It does not use a learner account, create real learner data or represent BYOL's production model.

8. Follow a contact enquiry: form, API, validation and providers

The form UI runs in a browser, but the server endpoint must independently verify every submission. Browser validation improves usability; server validation protects the service.



In the current configuration the contact form is disabled and explains why.

Step 1: visitor form

LeadForm.tsx displays name, email, intent, optional message, consent and a hidden bot-check field. If the intent is institution, it asks for an organization name. Labels and field-level messages help the visitor correct mistakes.

```
const payload = Object.fromEntries(new FormData(form).entries());
payload.consent = formData.get("consent") === "on";
const response = await fetch("/api/lead", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(payload),
});
```

FormData reads the fields. JSON.stringify turns them into the request body. fetch sends a POST request to the same site's API. The response tells the UI whether it succeeded or what needs correcting.

Step 2: Zod schema

src/lib/validation/lead.ts defines the exact allowed shape: trimmed name length 2-100, valid email up to 200 characters, one of four intent labels, optional organization/message with length limits, consent exactly true, and an optional honeypot. An extra rule requires organization for institution intent.

Step 3: the server route checks requests

Check in route.ts	Purpose
Only JSON content type	Avoid unexpected body formats.
Origin matches this site	Reject basic cross-site requests.
At most 10,000 bytes	Bound memory/work for a request.
Valid JSON object	Handle malformed or wrong-shaped input safely.
Honeypot empty	Identify common automated form submissions.
Zod safeParse succeeds	Validate on the trusted server, not only in the browser.
Feature/legal/provider settings exist	Refuse service until approved and configured.
Hash source IP + Upstash rate limit	Limit repeated requests without raw IP in the rate-limit key.
Resend sends plain text	Deliver email using server-only credentials.

Why the server is not the same as the browser

A visitor can change browser JavaScript or send requests without using the visible form. Therefore the API route repeats validation and checks approval, origin, size, bot field, provider setup and rate limit. Server secrets such as RESEND_API_KEY must never use a NEXT_PUBLIC_ prefix.

The form stays disabled by design: legal consent/privacy approval, Resend email settings and Upstash credentials are missing. Both visible form and API have separate gates; wait for owner approval before enabling.

9. How styling, mobile layout and accessibility work

The page markup names elements and classes. The CSS file gives those classes a visual appearance and changes the layout at smaller screen sizes.

CSS selectors and design variables

```
:root {
  --color-ink: #0b1b3b;
  --color-canvas: #faf7f0;
  --color-primary: #1f3a8a;
}

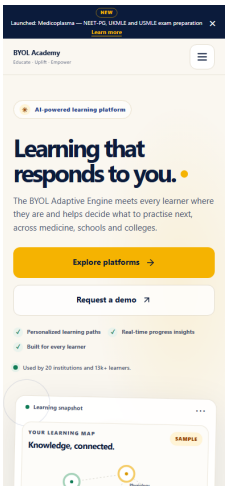
.shell {
  width: min(100% - 48px, 1200px);
  margin-inline: auto;
}

.button--primary {
  background: var(--color-accent);
}
```

A dot selector such as .shell targets an element whose className is shell. A CSS custom property begins with two hyphens; var() reuses it. The `.button--primary` class is added to button/link markup. One change to a shared variable can recolor the whole site.

Responsive design

The stylesheet has media queries near 1050, 820 and 600 pixels. A media query changes CSS only when the screen is within a size range. For example, the desktop navigation becomes a menu button on smaller screens, and multi-column card grids stack into fewer columns.



At phone width, the homepage changes its layout rather than shrinking every desktop element.

Accessibility is built into HTML and behavior

Implementation	Benefit
Skip to content link	Keyboard visitors bypass repeated navigation.
Button labels and aria-expanded	Assistive technology can identify menu controls and state.
Fieldset, legend, label	Questions and form inputs have meaningful grouping/names.
Focus moved to new demo heading	Keyboard users know when the question/results screen changes.
role=status / aria-live	Status and answer feedback can be announced.
SVG title/description plus data table	The learning map has text equivalents.
:focus-visible styles	Keyboard position is visible.
prefers-reduced-motion	Respects a system preference to reduce animation.

Automated axe checks found no serious/critical issues in the tested routes and states. This does not equal a full accessibility certification. A human should still test keyboard navigation, screen readers, zoom and real devices.

10. Metadata, search engines and browser security settings

Some project files generate descriptions for search engines, social sharing and crawlers. These improve clarity but do not guarantee search ranking.

File	What it generates
src/app/layout.tsx	Default browser/search title, description, canonical and social metadata.
src/app/opengraph-image.tsx	A 1200x630 share-preview graphic.
src/app/robots.ts	Crawler rules; preview sites can be completely blocked.
src/app/sitemap.ts	A list of public routes for search crawlers.
src/lib/seo.ts	Absolute URL and Organization/WebSite structured data.
src/components/JsonLd.tsx	Safe insertion of JSON-LD structured data.

Analytics is optional and gated

AnalyticsEvents.tsx watches page clicks and scroll depth only when enabled. analytics.ts also checks that the build is production and NEXT_PUBLIC_ANALYTICS_ENABLED is true. The intended event properties avoid names, emails and message contents. Confirm privacy notice and vendor configuration before enabling.

Security headers

next.config.ts adds HTTP response headers: HSTS for HTTPS, nosniff, referrer policy, a restricted permissions policy and a frame-denial rule. These are browser instructions; they do not replace safe server logic, dependency updates, or deployment checks.

Crawler files are not pages visitors normally browse

robots.ts becomes /robots.txt. sitemap.ts becomes /sitemap.xml. Next.js understands those special file names and generates the matching responses.

11. How tests and continuous integration protect changes

A test records a behavior we expect. When code changes, rerunning it helps catch accidental breakage.

Test file	What it checks
tests/unit/mastery.test.ts	Correct/incorrect answer updates, probability bounds and labels.
tests/unit/content.test.ts	Approved content filtering and sample data shape.
tests/unit/lead-validation.test.ts	Valid/invalid lead input and institution condition.
tests/unit/lead-route.test.ts	Request type/origin/size/honeypot/configuration protections.
tests/e2e/site.spec.ts	Real routes, mobile menu, demo, horizontal overflow and axe.

Unit test versus browser test

A unit test calls a small function directly; for example, it checks that a correct answer raises the demo estimate. An end-to-end browser test opens the rendered website, finds controls by accessible names, clicks through the demo and checks the final table.

The main local checks

```
npm run lint       # code-quality and common mistake checks
npm run typecheck  # TypeScript checks without output files
npm test           # unit tests through Vitest
npm run test:e2e   # browser tests through Playwright
npm run build      # production build
```

What the tagged CI workflow does

```
name: ci
on:
  pull_request:
  push:
    branches: [main]
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm run lint
      - run: npm run typecheck
      - run: npm test
      - run: npm run build
      - run: npx playwright install --with-deps chromium
      - run: npm run test:e2e
      - run: npm audit --audit-level=high
```

Workflow line	In ordinary language
pull_request / push to main	Run checks when someone proposes a change or pushes to main.
ubuntu-latest	Use a clean temporary Linux machine.
checkout	Download the source code to that machine.
setup-node 22 + npm cache	Install the intended Node version and reuse safe package downloads.
npm ci	Install exactly what package-lock.json records; fail if lock and manifest disagree.
lint/typecheck/test/build	Run project quality gates in a repeatable order.

Workflow line	In ordinary language
Playwright browser install/tests	Install Chromium dependencies and run real-browser scenarios.
npm audit	Fail for high-severity known dependency advisories.

CI means continuous integration: an automated teammate repeats checks on a clean machine. This workflow verifies code; it does not publish the website, provide legal approval, or confirm production credentials.

12. Root files and configuration: the project control panel

Root files sit beside src and tests. They tell tools what the project is, how to compile it, and how to run checks.

Root file/folder	Purpose
package.json	Project identity, pinned dependencies and npm scripts.
package-lock.json	Exact full dependency tree used by npm install/ci.
tsconfig.json	TypeScript strictness, browser libraries, JSX handling and @/* alias.
next.config.ts	Next.js settings and security response headers.
eslint.config.mjs	ESLint rules based on Next core web vitals and TypeScript.
vitest.config.ts	Unit test discovery, React transform and @ alias.
playwright.config.ts	Browser test port, mobile/desktop projects, retries and traces.
next-env.d.ts	Generated Next.js type references. Usually do not edit.
.gitignore	Files/folders Git should not record, including generated output and secrets.
.env.example	Names the environment variables and safe off/blank defaults.
README.md	Setup, local commands and deployment environment notes.
LAUNCH_CHECKLIST.md	Requirements R1-R15, their status and business launch dependencies.
AGENTS.md	Instructions for coding-agent tools; not part of the public website.
.github/workflows/ci.yml	Automated verification on pull requests and main pushes.

package.json: scripts versus dependencies

Dependencies are libraries the application imports, such as Next.js, React, Zod and Resend. Development dependencies are tools for building or testing, such as TypeScript, ESLint, Vitest and Playwright. The scripts are short commands with friendly names.

```
"scripts": {
  "dev": "next dev",
  "build": "next build",
  "start": "next start",
  "lint": "eslint .",
  "typecheck": "tsc --noEmit",
  "test": "vitest run",
  "test:e2e": "playwright test"
}
```

Why there is a lock file

package.json says which direct packages the app wants. Those packages also depend on other packages. package-lock.json records the resolved versions of that whole tree so a fresh computer can install the same versions. Do not hand-edit the lock file; npm updates it when dependencies change.

Why tsconfig uses an alias

The compiler option paths maps @/* to ./src/*. Thus @/components/Header means src/components/Header. Strict type checks and noUncheckedIndexedAccess help reveal missing values.

13. Environment variables, documents and generated files

Configuration varies by computer and deployment. Environment variables keep settings out of the source code, and secret values stay on the server.

Variable	What it controls	Initial state
NEXT_PUBLIC_SITE_URL	Canonical public site URL.	Example domain in .env.example.
NEXT_PUBLIC_ANALYTICS_ENABLED	Whether browser analytics is included.	false
NEXT_PUBLIC_LEAD_FORM_ENABLED	Whether to display the form.	false
LEAD_FORM_ENABLED	Server-side lead endpoint gate.	false
LEAD_CONSENT_APPROVED	Legal approval gate.	false
RESEND_API_KEY / LEAD_TO_EMAIL / LEAD_FROM_EMAIL	Private email delivery settings.	blank
UPSTASH_REDIS_REST_URL / TOKEN	Private rate-limit connection.	blank

NEXT_PUBLIC is not for secrets
A NEXT_PUBLIC_ value may be included in code sent to the browser. Never put passwords, API keys or private tokens in a NEXT_PUBLIC_ variable.

Supplied reference documents

Root document	How it relates to the build
BYOL_Academy_Agent_Build_Guide.pdf	Original website requirements and guardrails.
Adaptive_Learning_Engines_Research_Brief.docx	Research background for cautious adaptive-learning explanations.
Knewton.pdf	Additional supplied reference material; not website source code.
BYOL_Academy_Beginner_Guide.pdf	Earlier website walkthrough with screenshots and a broad beginner tour.
BYOL_Academy_Code_Explained.pdf	Shorter code-focused companion guide.

Generated and local-only items

node_modules is downloaded code. .next and tsconfig.tsbuildinfo are generated caches. They are recreated by npm/Next.js and should not be treated as authored source. .gitignore excludes these and private .env files. The .env.example file is safe to share because its secret values are blank.

Folder versus Git repository
The folder is a project workspace. A Git repository adds a .git history folder and tracks changes/commits. The current workspace is not initialized as Git, so these files can still be edited and run, but local version history is not active.

14. Complete source-file catalog - app and components

This is a concise explanation of every authored source file found under src/app and src/components. Use Ctrl+P in VS Code to open a filename quickly.

File	What it is for
src/app/layout.tsx	Shared HTML document, global CSS import, Header/Footer, metadata and gated analytics.
src/app/page.tsx	Homepage sections and data-driven products/audiences.
src/app/about/page.tsx	About introduction, conceptual engine loop, research and principles.
src/app/blog/page.tsx	Blog landing/empty state; no articles are fabricated.
src/app/contact/page.tsx	Reads intent query; renders contact information and LeadForm.
src/app/privacy/page.tsx	Noindex placeholder; explicitly says it is not a Privacy Policy.
src/app/terms/page.tsx	Noindex placeholder until legal terms are approved.
src/app/styleguide/page.tsx	Noindex palette/button/type/status preview.
src/app/not-found.tsx	Branded page for an unknown URL.
src/app/error.tsx	Client-side retry page for rendering errors; logs safe error identifier.
src/app/api/lead/route.ts	POST handler for lead validation, request checks, rate limiting and email.
src/app/opengraph-image.tsx	Generated image used when a page is shared.
src/app/robots.ts	Creates crawler rules and points to sitemap.

File	What it is for
src/app/sitemap.ts	Creates sitemap entries for public routes.
src/components/Header.tsx	Responsive navigation, mobile state, Escape behavior, external platform menu.
src/components/AnnouncementBar.tsx	Dismissible message with tab-session persistence.
src/components/Footer.tsx	Shared product, company and legal links.
src/components/AdaptiveDemo.tsx	Subject selection, quiz states, feedback, estimates and result UI.
src/components/MasteryMap.tsx	SVG map; accepts estimates/concepts through props and includes text table.
src/components/LeadForm.tsx	Form UI, states, form submission and clear disabled state.
src/components/AnalyticsEvents.tsx	Optional delegated click and scroll-depth listeners.
src/components/ApprovalNotice.tsx	Development notice for copy/data awaiting approval.
src/components/JsonLd.tsx	Safely serializes structured JSON for search crawlers.

15. Complete source-file catalog - data, logic, tests and your first lessons

The remaining source files are where information, calculations and expected behavior live. The final section gives you small exercises in a safe order.

File	What it is for
src/content/site.ts	Brand, site copy, navigation, typed platforms/audiences/benefits/stats/principles and approval helper.
src/content/demo-data.ts	Question objects for three demo subjects.
src/lib/adaptive/mastery.ts	Illustrative probability calculation and readable status labels.
src/lib/validation/lead.ts	Zod schema describing acceptable lead form data.
src/lib/analytics.ts	One gated wrapper around Vercel analytics events.
src/lib/seo.ts	Canonical URL helper and organization/website structured data.
src/styles/globals.css	Design tokens, common UI styles, breakpoints, focus and reduced motion.
tests/unit/mastery.test.ts	Behavior of mastery update and boundaries.
tests/unit/content.test.ts	Content approval and demo-data conditions.
tests/unit/lead-validation.test.ts	Lead schema valid/invalid cases.
tests/unit/lead-route.test.ts	API route safety and configuration cases.
tests/e2e/site.spec.ts	Browser routes, demo, mobile menu, overflow and accessibility.

Exercise 1: change a safe piece of wording

Open src/content/site.ts. Find the hero description inside copy.hero. Change one sentence without changing the object structure. Save, view the browser, then run npm run lint and npm run typecheck.

Exercise 2: follow one product card

Find the platforms array in site.ts. Choose a product's name and summary, then use Ctrl+Shift+F to search the source for `platform.summary`. Follow the import in app/page.tsx and observe how the same object supplies the card.

Exercise 3: follow a demo question

Open demo-data.ts and locate the Medicine question list. Identify prompt, options, answer and explanation. The answer field is a zero-based index: 0 is the first option, 1 the second, and so on. Run npm test after making a carefully checked sample-content edit.

Exercise 4: inspect responsive CSS

Open the site in a browser and resize it. Find @media in globals.css. Compare the navigation and card layouts at wide and narrow sizes. Change only a small spacing/color value, then inspect both desktop and phone widths.

Your change-and-check routine

```
1. Read the relevant source and nearby content.
2. Make one small change.
3. Save and inspect the local browser.
4. Run npm run lint
5. Run npm run typecheck
6. Run npm test (and browser tests for UI changes).
7. Review the final screen and the terminal output.
```

A beginner glossary

Word	Meaning
Route	A URL path such as /about.
Component	Reusable React interface function.
Props	Inputs passed into a component.
State	Data a component remembers between interactions.
Render	Create the output shown on screen.
API	A request endpoint handled by server code.
Validation	Checking that input follows expected rules.
Dependency	A library the project uses.
Build	Create optimized production output.
Test	Executable check of expected behavior.
CI	Automated checks run when code is proposed/pushed.
SEO	Page structure/metadata that helps search systems understand content.
Accessibility	Making a site usable by people with different abilities and input methods.

What to learn next
Learn HTML structure, CSS layout, JavaScript variables/functions/arrays, then TypeScript types and React props/state. This project can be your map: first read, then make tiny edits, then build a small page of your own.

End of learning guide. Website status and launch dependencies are tracked in LAUNCH_CHECKLIST.md.